

Criterion C

Figma was used for interface wireframes, while Mermaid was used for architecture, database and algorithm diagrams.

0. System model

Component / Architecture Diagram

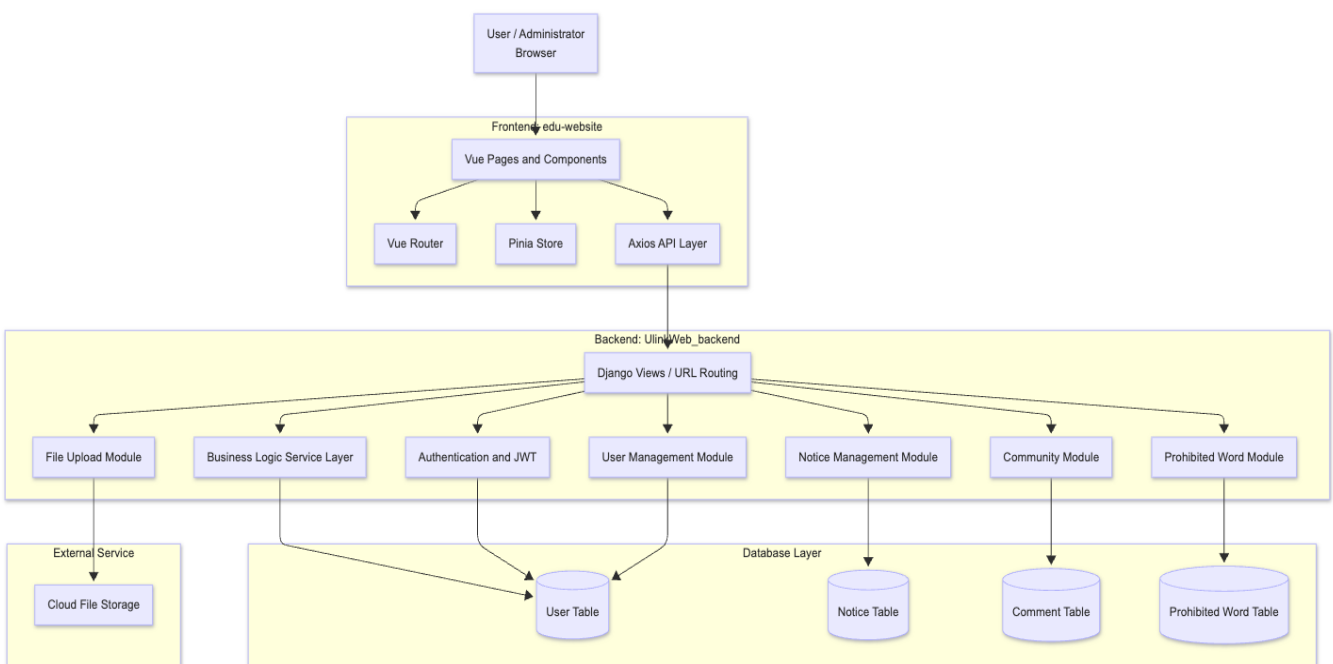


Fig. 0.1

Figure 0.1 shows a layered design: Vue handles presentation and client state, Axios sends API requests, Django applies authentication and business rules, and MySQL or cloud storage persists records and uploaded images.

DFD Level 0 (Context Diagram)

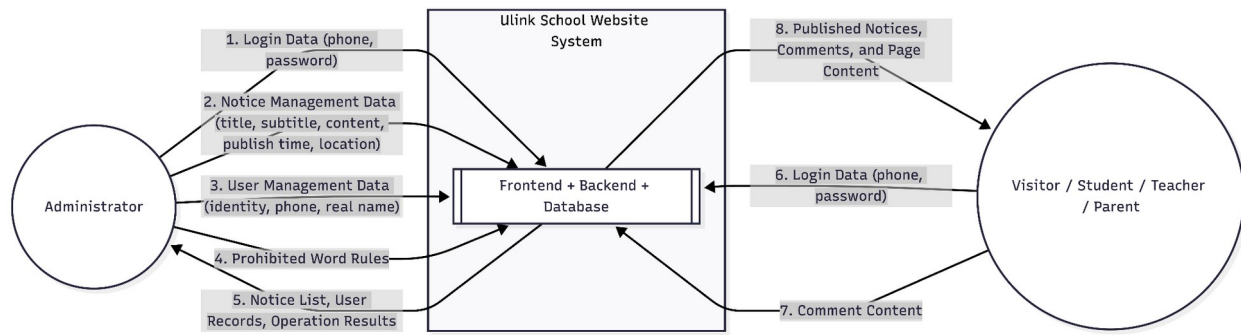


Fig. 0.2

UI Navigation Map

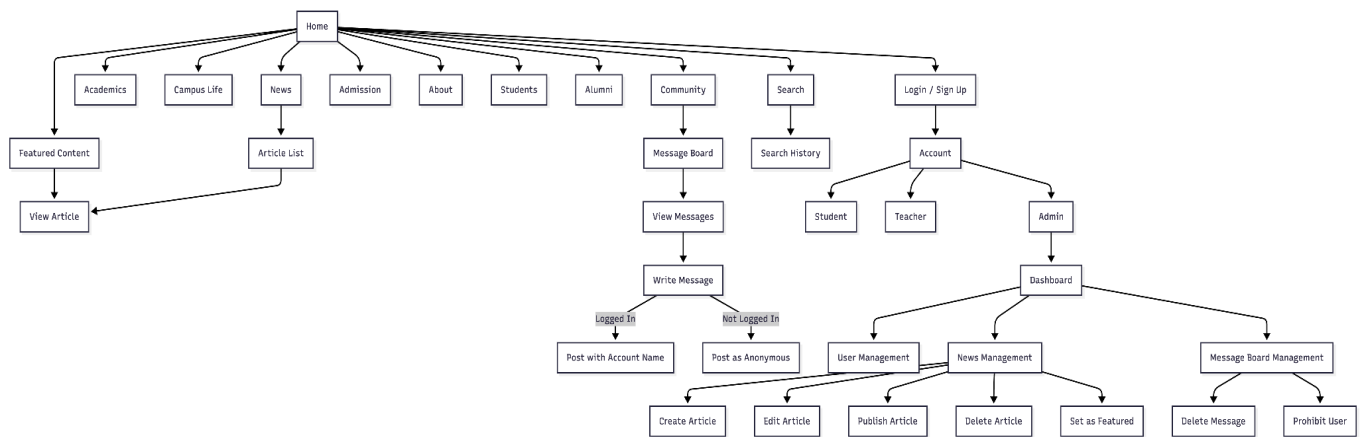


Fig. 0.3

Figures 0.2 and 0.3 define the data exchanged with administrators and public users, then separate public pages from authenticated community actions and role-restricted dashboard functions.

1. Database Schema

Table: admin_role_notice

admin_role_notice					
Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
title	varchar(255)	NO		NULL	
subtitle	varchar(255)	YES		NULL	
content	text	YES		NULL	
publisher	varchar(255)	NO		NULL	
status	varchar(255)	NO		NULL	
publish_time	datetime	NO		NULL	
create_time	datetime	NO		NULL	
cover	varchar(255)	NO		NULL	
user_id	int(20)	NO	MUL	NULL	

Fig. 1.1

Table: admin_notice_copy1

admin_notice_copy1					
Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
title	varchar(255)	NO		NULL	
subtitle	varchar(255)	YES		NULL	
content	text	YES		NULL	
publisher	varchar(255)	YES		NULL	
status	varchar(255)	NO		NULL	
publish_time	datetime	YES		NULL	
create_time	datetime	NO		NULL	
cover	varchar(255)	NO		NULL	
user_id	int(20)	YES	MUL	NULL	
publish_location	varchar(255)	NO		NULL	
position_index	varchar(255)	YES		NULL	

Fig. 1.2

Table: admin_role_admin

admin_role_admin					
Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
user_name	varchar(255)	NO		NULL	
real_name	varchar(255)	NO		NULL	
phone	varchar(255)	NO		NULL	
password	varchar(255)	NO		NULL	
identity	varchar(255)	NO		student	
create_time	datetime	NO		NULL	
update_time	datetime	YES		NULL	

Fig. 1.3

Table: admin_role_teacher

admin_role_teacher					
Field	Type	Null	Key	Default	Extra
id	int(11)	NO	Primary	NULL	auto_increment
teacher_name	varchar(20)	NO		NULL	
gender	tinyint(1)	NO		NULL	
phone	varchar(255)	NO		NULL	
address	varchar(255)	NO		NULL	
email	varchar(255)	NO		NULL	
work_code	varchar(255)	NO		NULL	
subject_name	varchar(255)	NO		NULL	
subject_id	int(11)	YES		NULL	
create_time	datetime	NO		NULL	
create_by	varchar(255)	NO		NULL	
create_id	int(11)	NO		NULL	
update_time	Datetime	YES		NULL	
update_by	varchar(255)	YES		NULL	
update_id	int(11)	YES		NULL	

Fig. 1.4

Table: admin_role_students

admin_role_students

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	Primary	NULL	auto_increment
name	varchar(20)	NO		NULL	
age	int(3)	NO		NULL	
gender	tinyint(1)	NO		NULL	
phone	varchar(20)	NO		NULL	
stu_numb	varchar(50)	NO		NULL	
class_id	int(11)	NO		NULL	
address	varchar(255)	NO		NULL	
create_time	datetime	NO		NULL	
create_by	varchar(255)	NO		NULL	
create_id	int(11)	NO		NULL	
update_time	Datetime	YES		NULL	
update_by	varchar(255)	YES		NULL	
update_id	int(11)	YES		NULL	

Fig. 1.5

Table: community_comment

community_comment

Field	Type	Null	Key	Default	Extra
id	bigint(20)	NO	Primary	NULL	auto_increment
content	longtext	NO		NULL	
author	varchar(100)	NO		NULL	
created_at	datetime(6)	NO		NULL	

Fig. 1.6

ERD Diagram

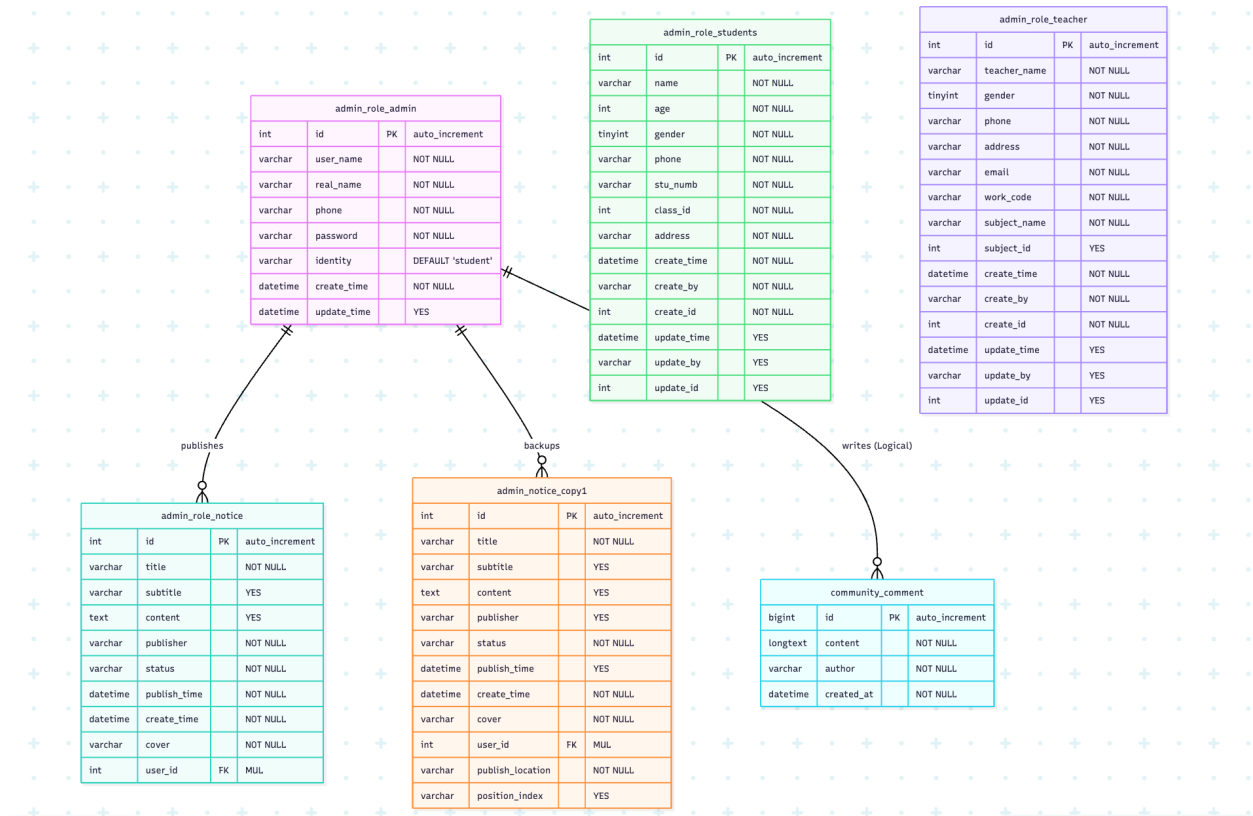


Fig. 1.7

Figure 1.7 shows the notice publishing and backup relationships. Comment authorship is logical because the authenticated display name is stored in the comment rather than through a physical foreign key.

2. The School Official Website consists of six main components:

1. Authentication & Role Module (Success Criterion 1)

- Wireframe of GUI: Login and registration forms collect phone, password and account details.
- Function Description: Compares the MD5-derived credential and issues a JWT containing the authenticated user's identity.

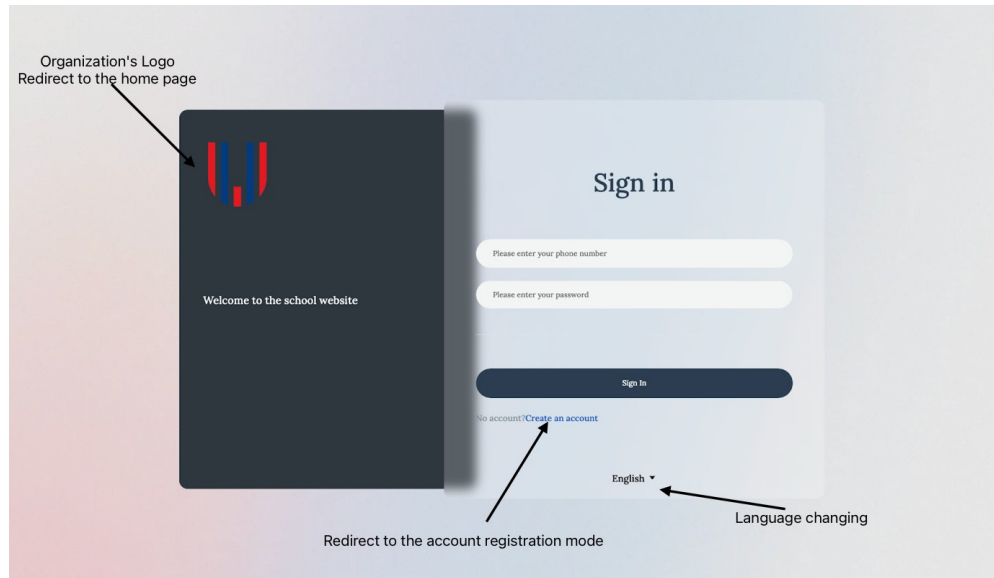


Fig. 2.1.1

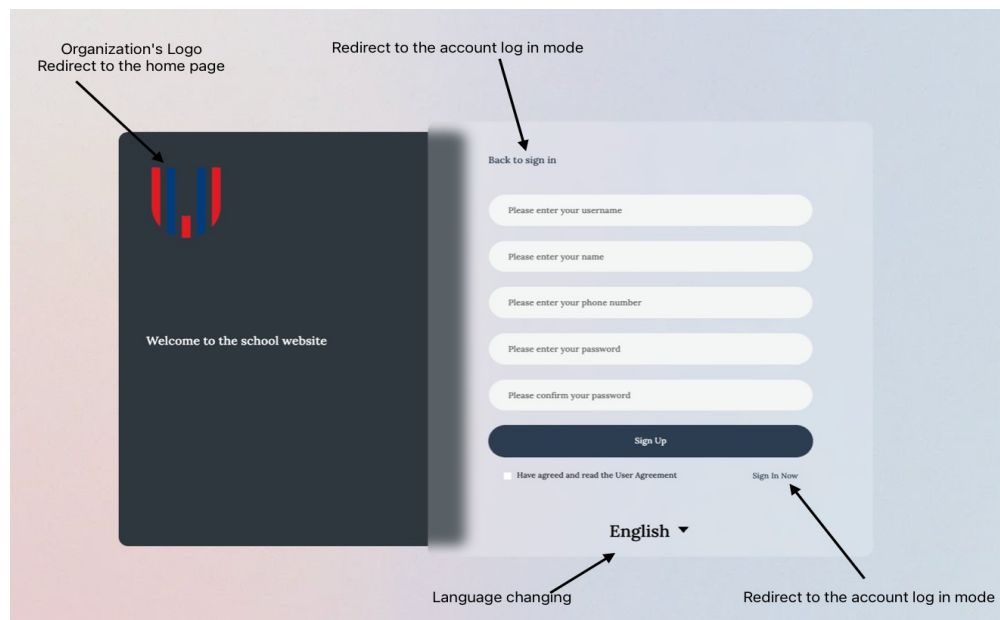


Fig. 2.1.2

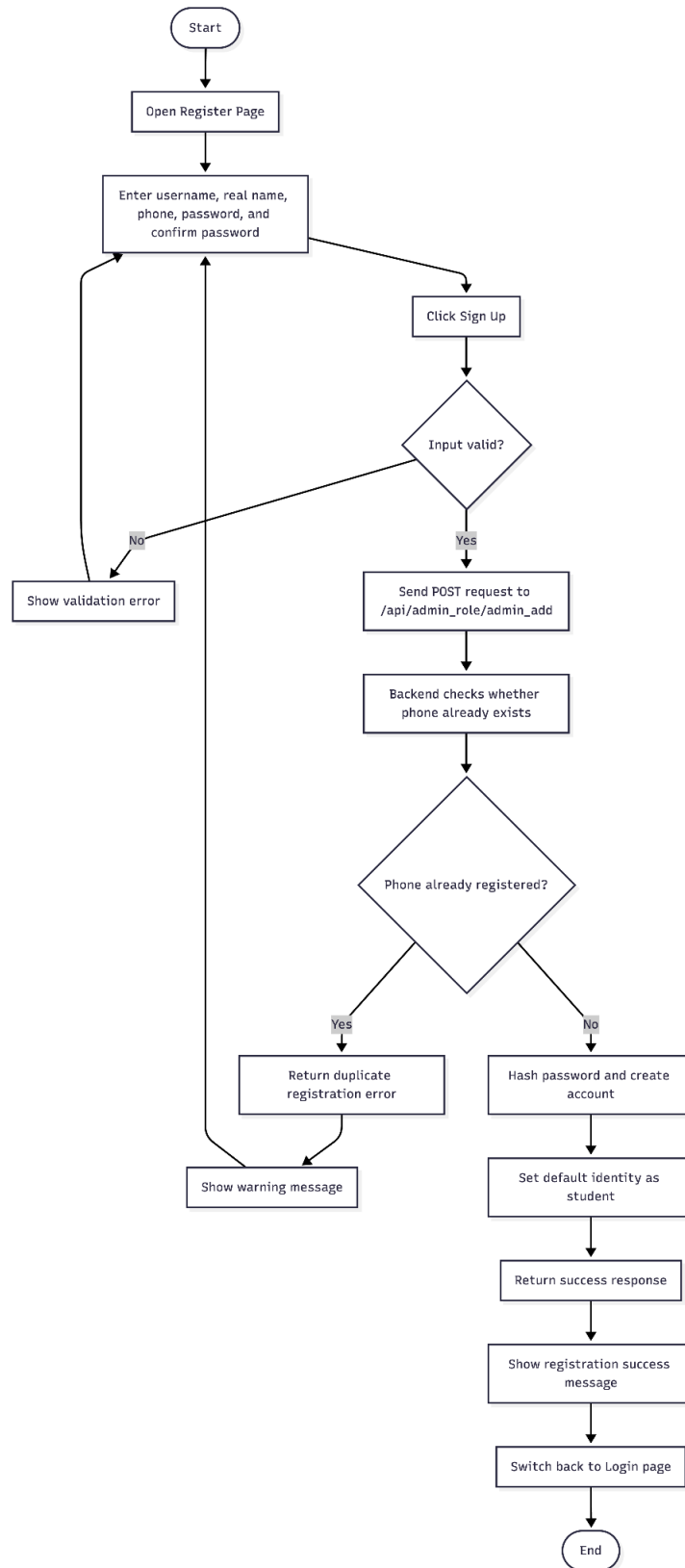


Fig. 2.1.3 Register

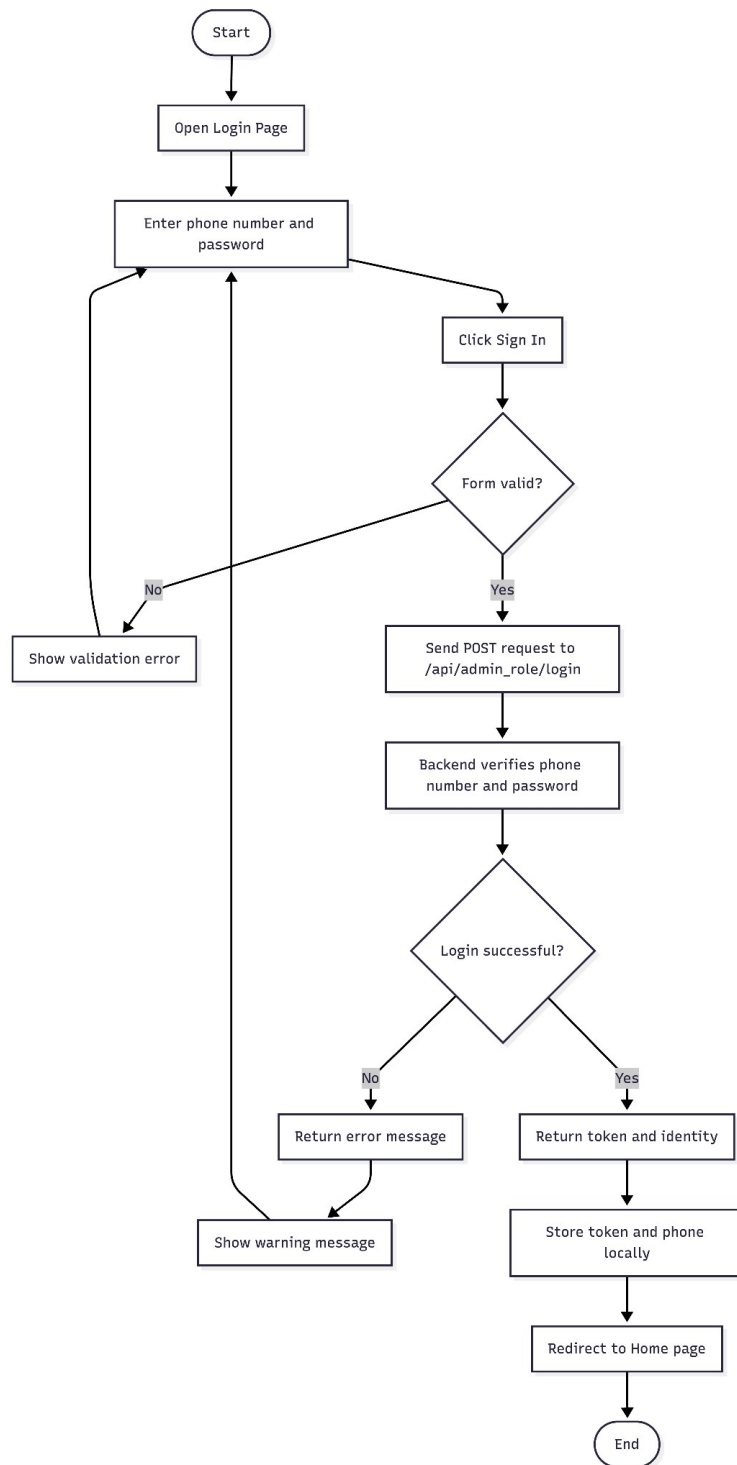


Fig. 2.1.4 Login

2. Notice Management Module (Success Criterion 2)

- Wireframe of GUI: Admin dashboard with "Create" button and "Edit/Delete/Withdraw" actions.
- Function Description: Provides full CRUD operations for school notices and allows manual status changes.
- Algorithm (Flowchart Logic): Input Notice Data -> Validate Permission -> SQL: UPDATE/INSERT Notice Table -> Return Success -> Update UI.

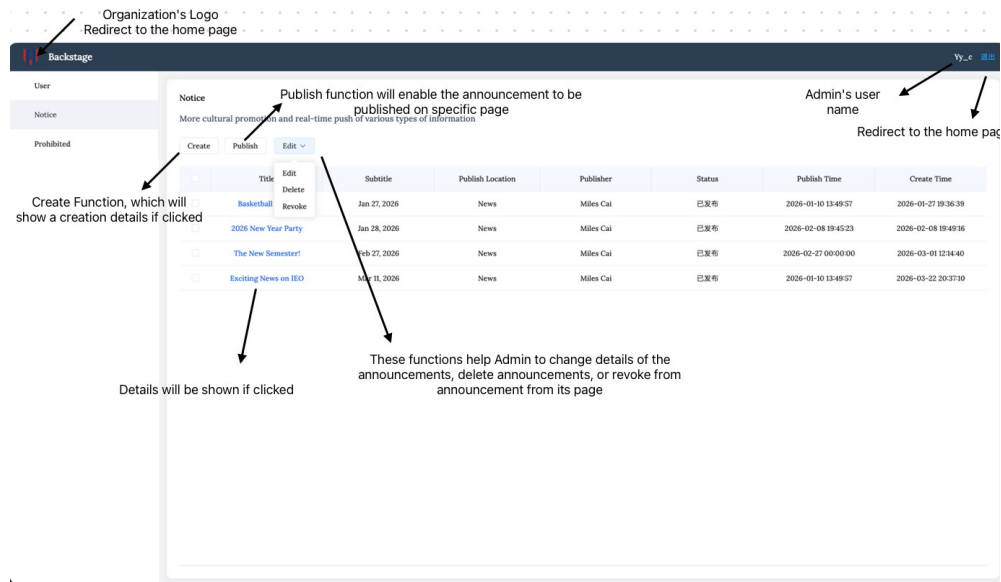


Fig. 2.2.1

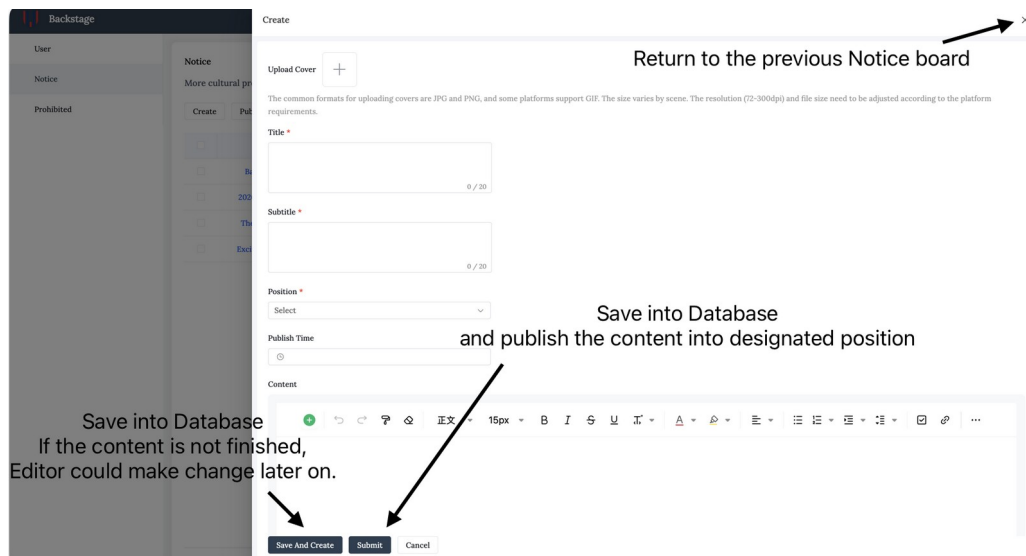


Fig. 2.2.2

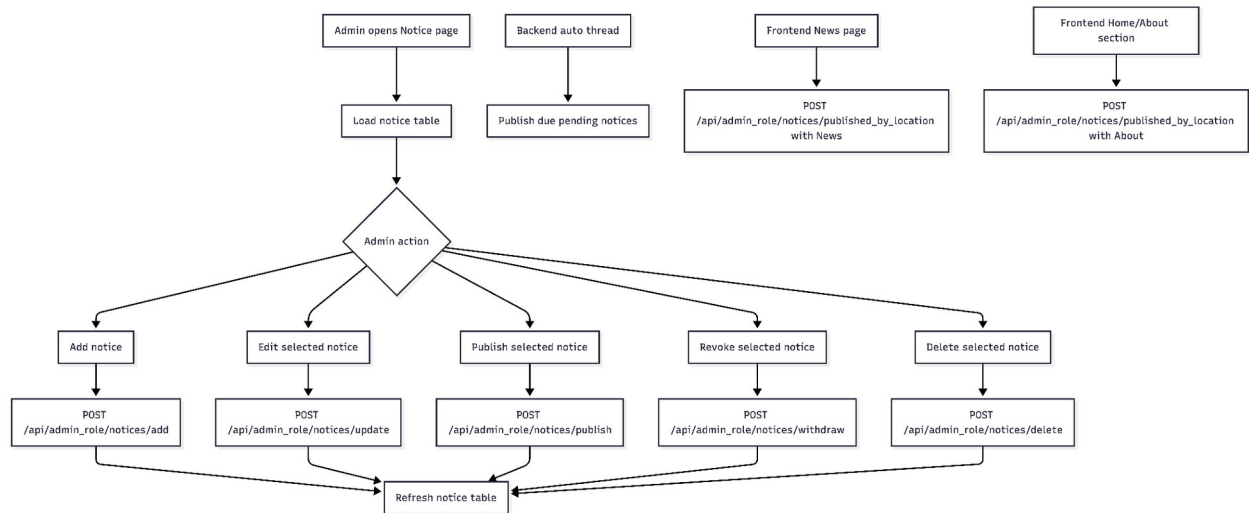


Fig. 2.2.3 Overall Module Structure

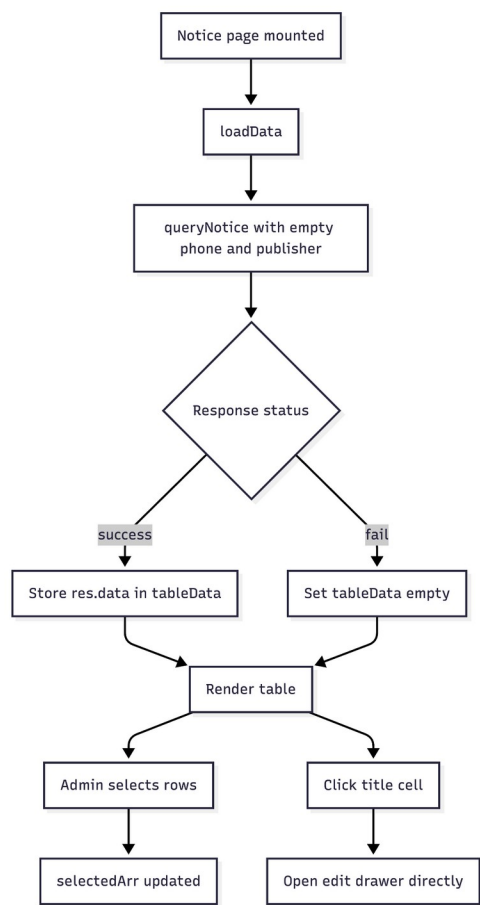


Fig. 2.2.4 Admin List And Select Notices

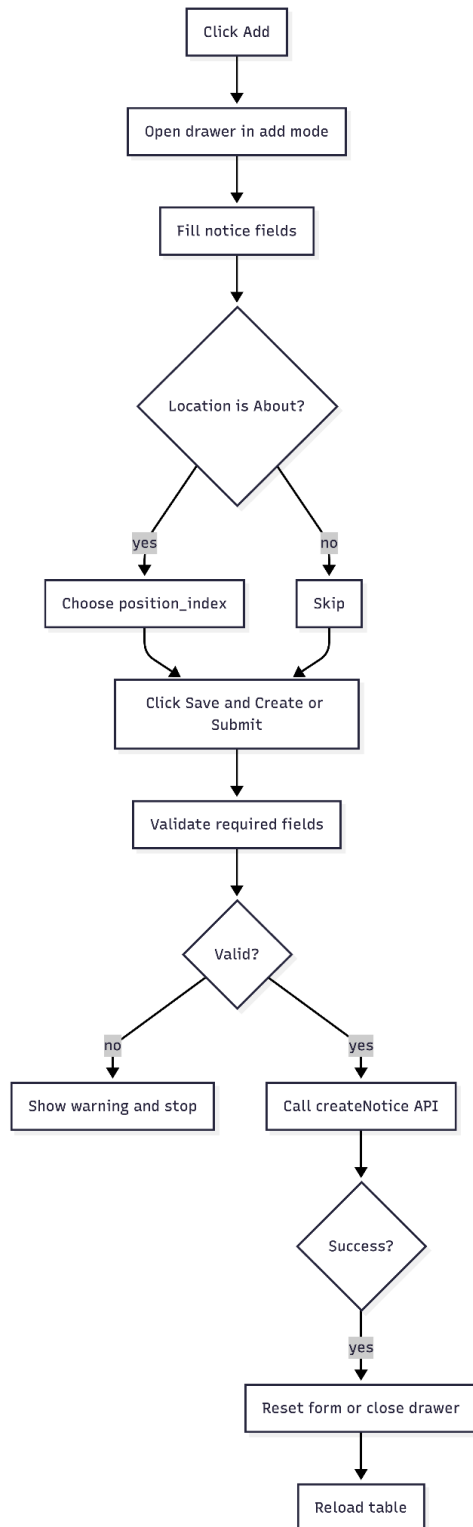


Fig. 2.2.5 Add Notice: Frontend Form

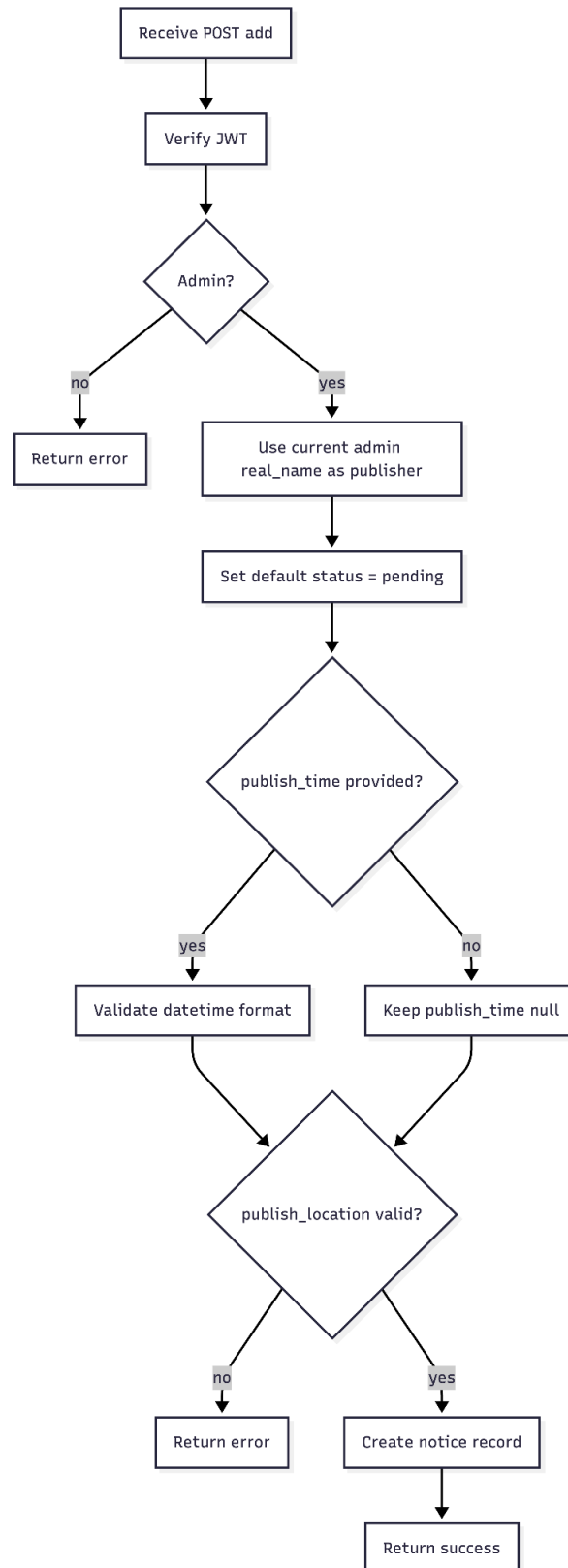


Fig. 2.2.6 Add Notice: Backend Creation

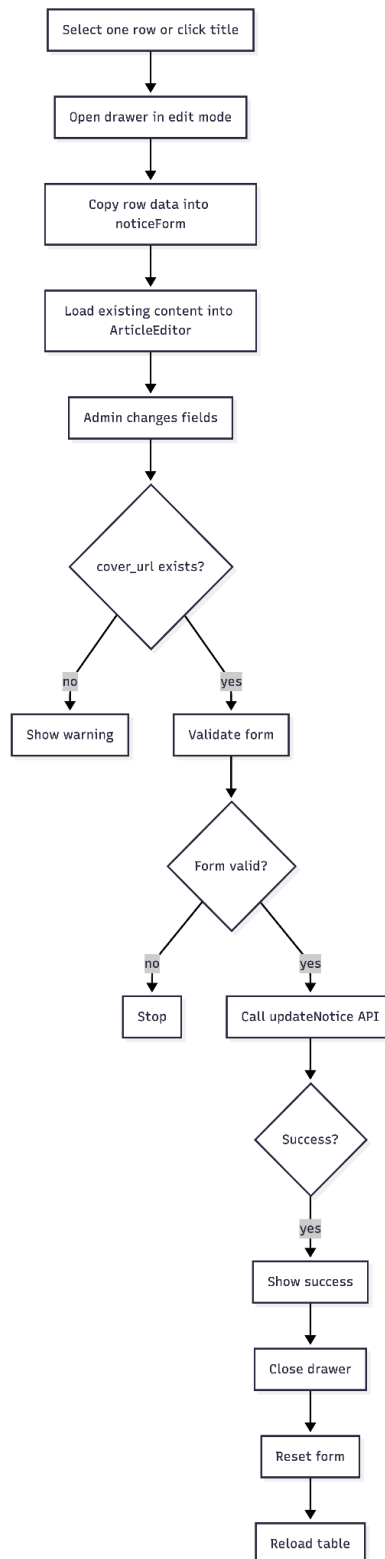


Fig. 2.2.7 Edit Notice: Frontend Submit

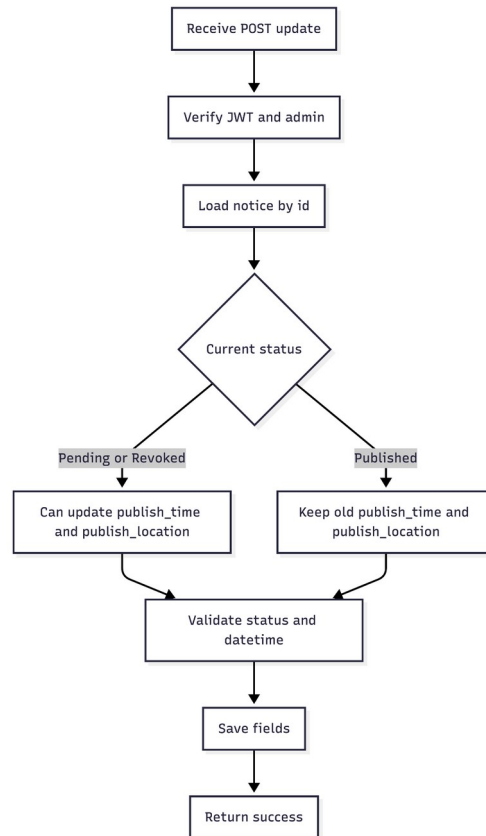


Fig. 2.2.8 Edit Notice: Backend Status Rules

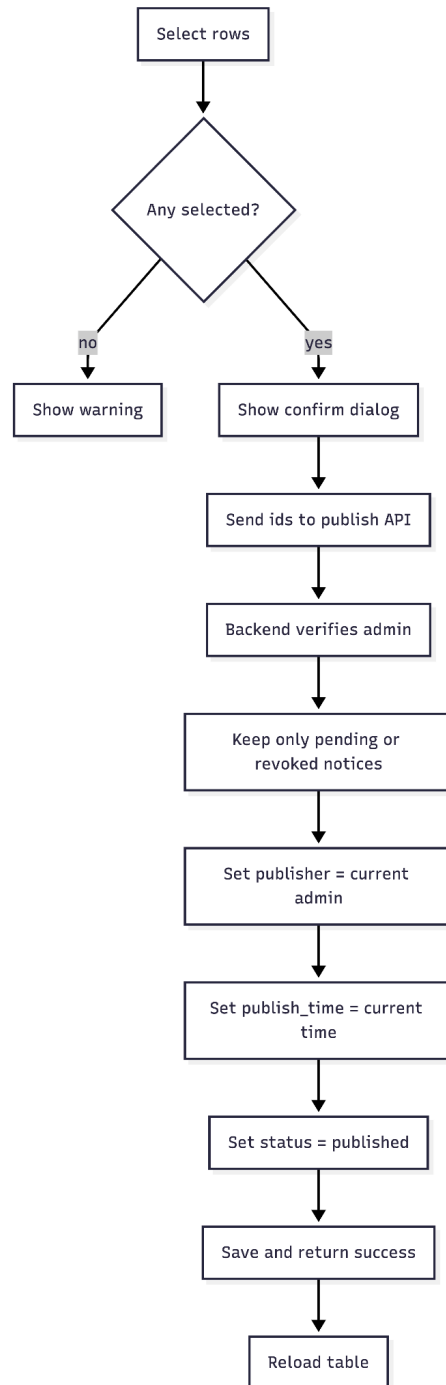


Fig. 2.2.9 Publish Notice

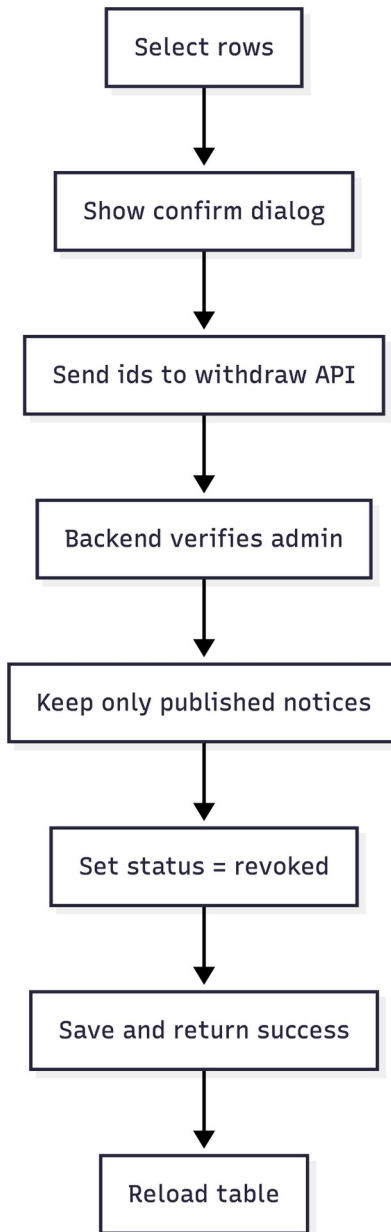


Fig. 2.2.10 Revoke Notice

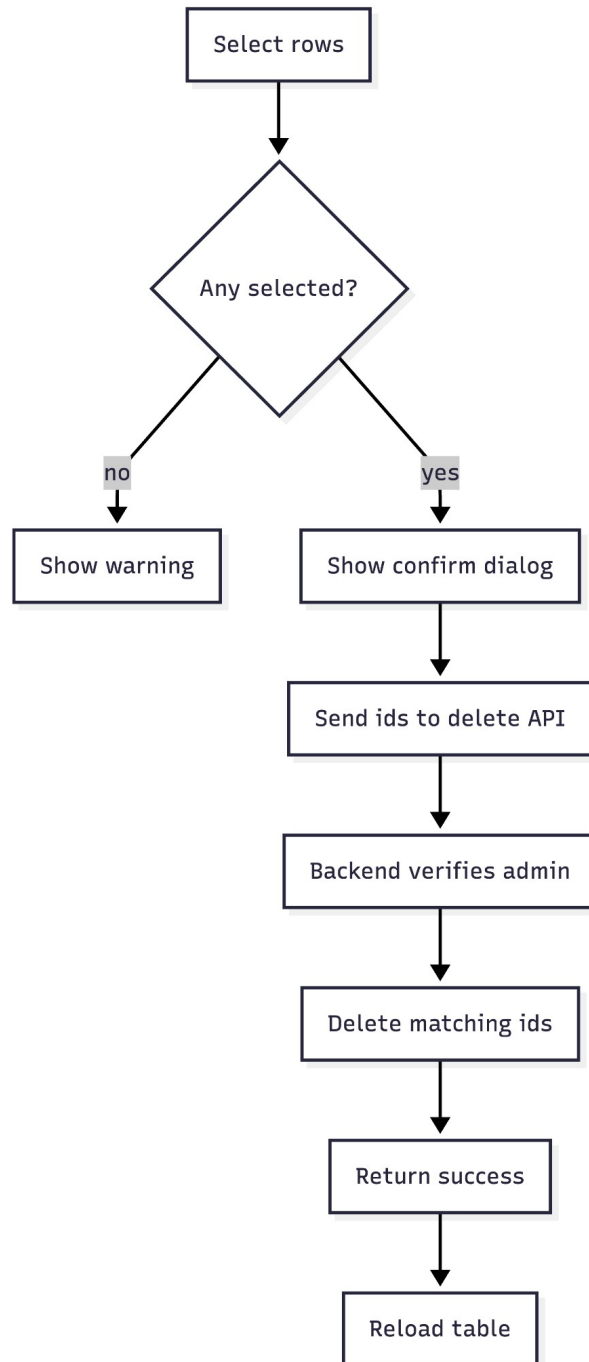


Fig. 2.2.11 Delete Notice

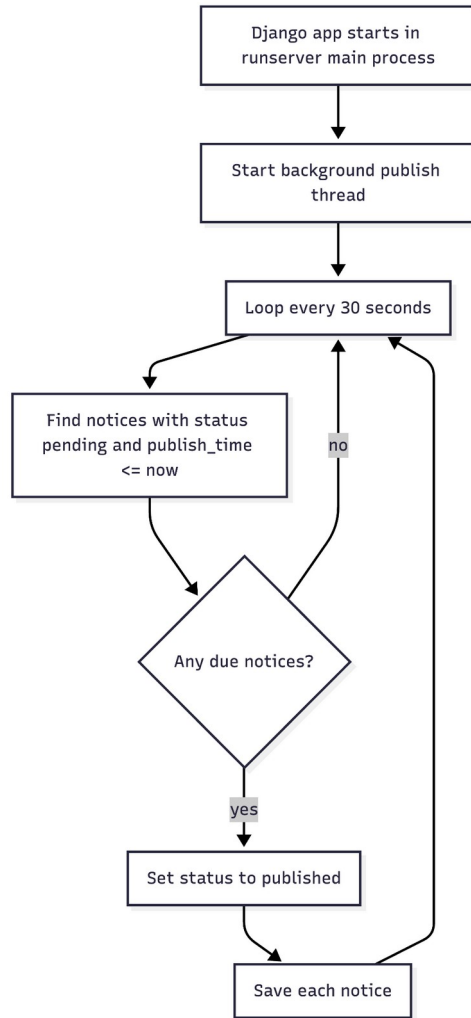


Fig. 2.2.12 Backend Auto-Publish Flow

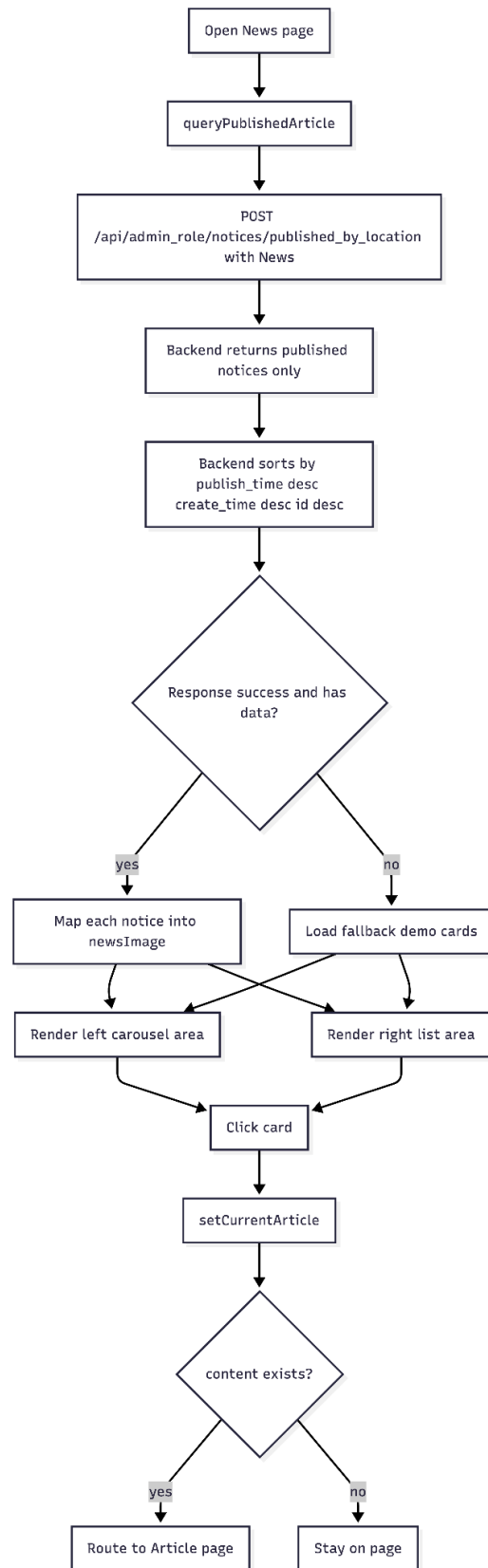


Fig. 2.2.13 Frontend News Page Rendering

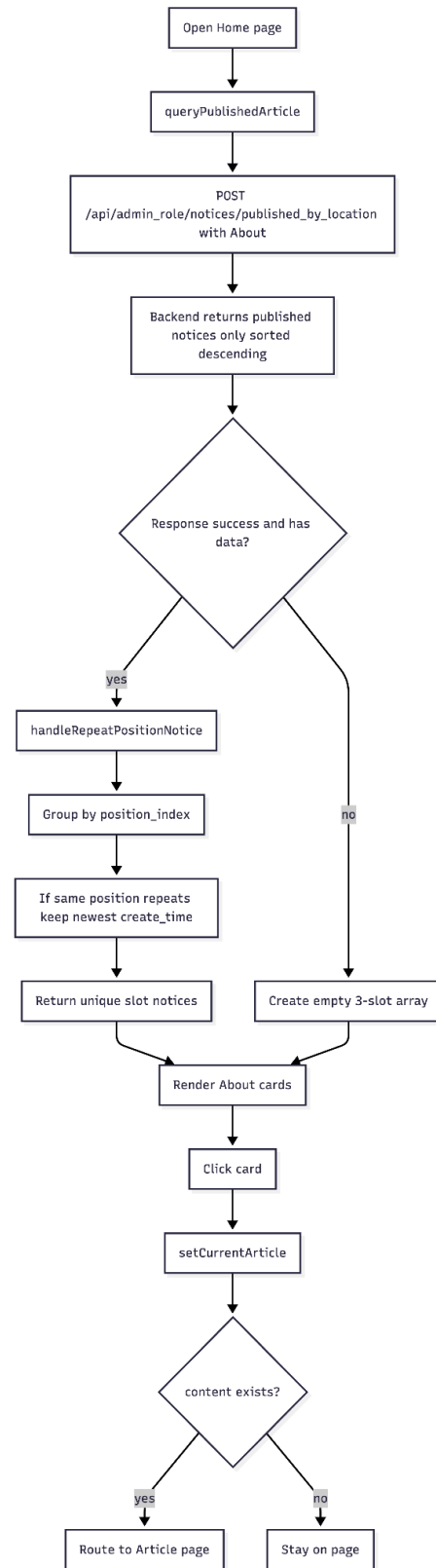


Fig. 2.2.14 Frontend About Page Slot Rendering

3. Automated Publishing Module (Success Criteria 2 and 3)

- Wireframe of GUI: Date-picker on the edit page for "Scheduled Publish Time".
- Function Description: A background thread changes pending notices to published when their scheduled time is reached.

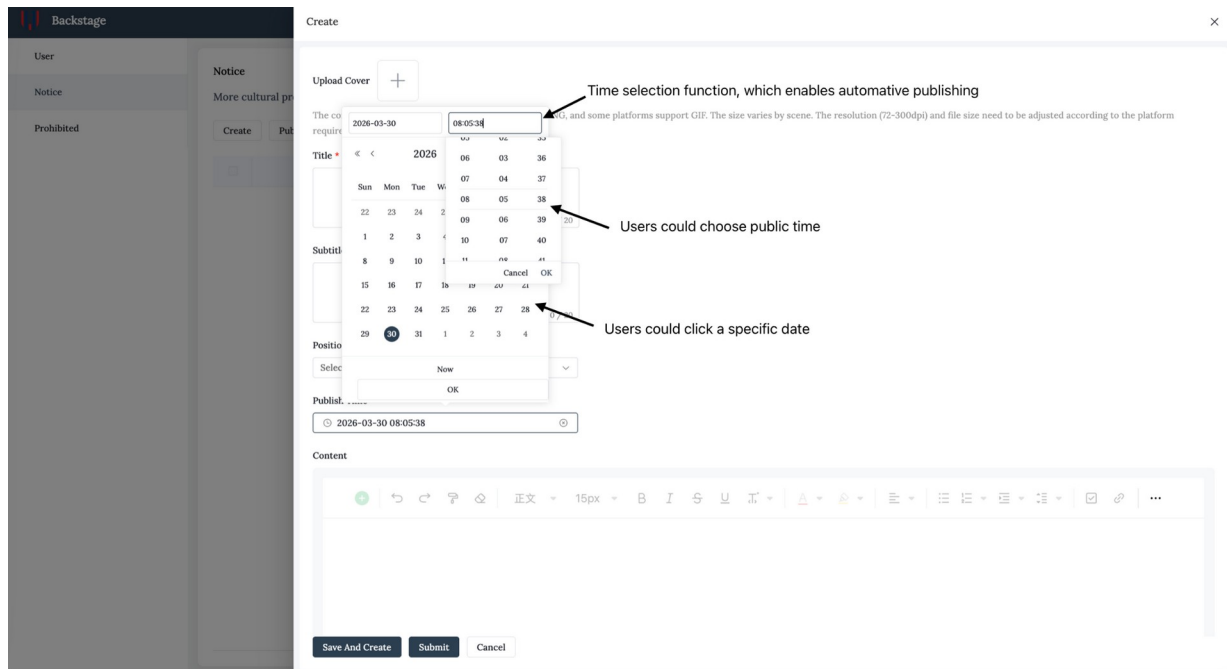


Fig. 2.3.1

Pseudocode for this function:

BEGIN Automated_Publishing_Module

Start module only when the backend server runs in runserver mode

Create a background thread

WHILE true DO

current_time <- get current timezone-aware system time

notice_list <- query database for notices

where status = Pending

and publish_time <= current_time

FOR each notice in notice_list DO

```
notice.status <- Published
```

```
save notice
```

```
ENDFOR
```

```
sleep for 30 seconds
```

```
ENDWHILE
```

```
END Automated_Publishing_Module
```

4. Community Interactive Module (Success Criterion 4)

- Wireframe of GUI: Discussion feed with comment cards and a text submission box.
- Function Description: DRF validates comment data; the backend verifies the JWT and stores the authenticated user's display name as author.
- Algorithm (Flowchart Logic): Student Submits -> Serializer Validation -> Attach Author Name -> SQL: INSERT Comment -> Refresh Feed.

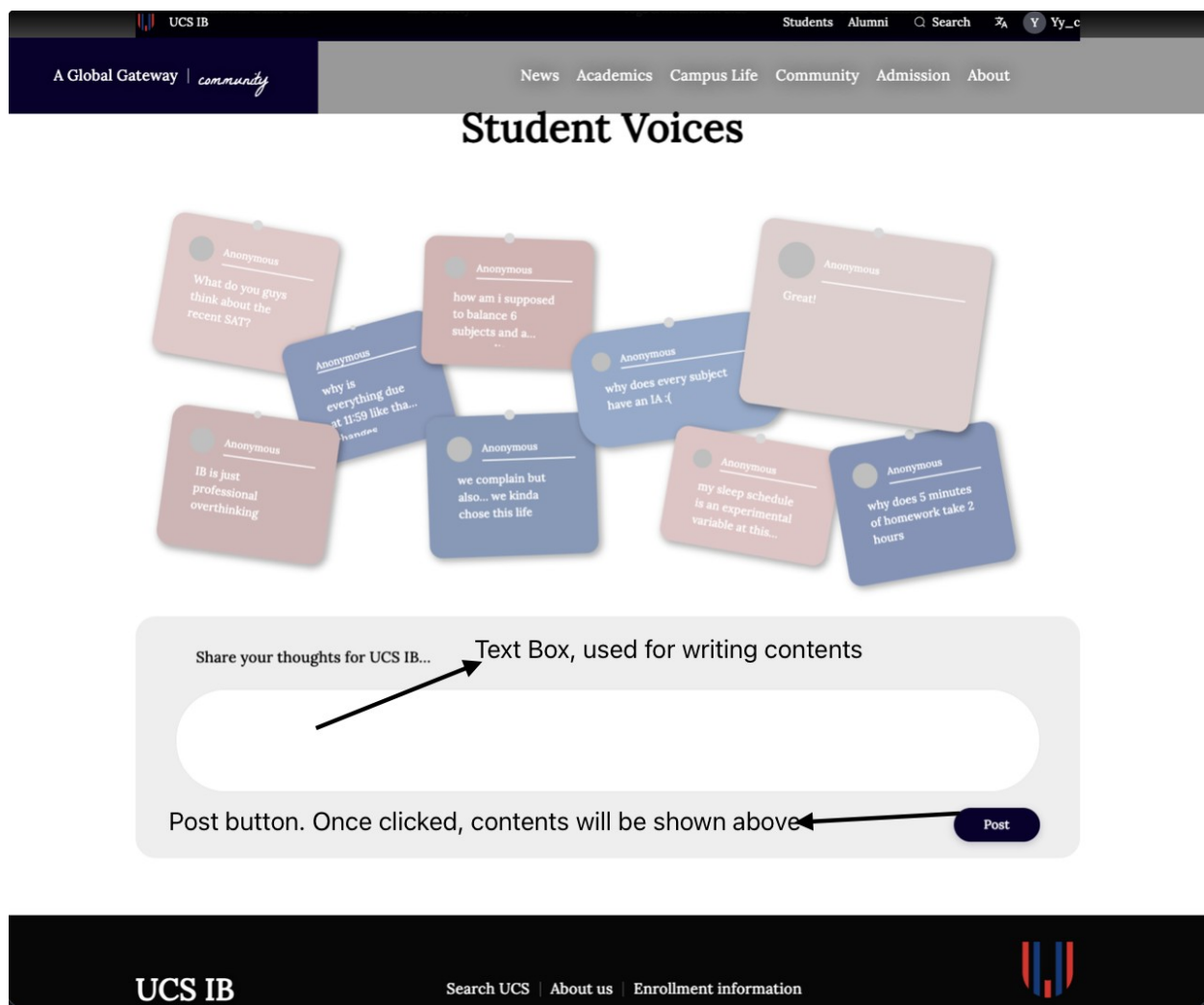


Fig. 2.4.1

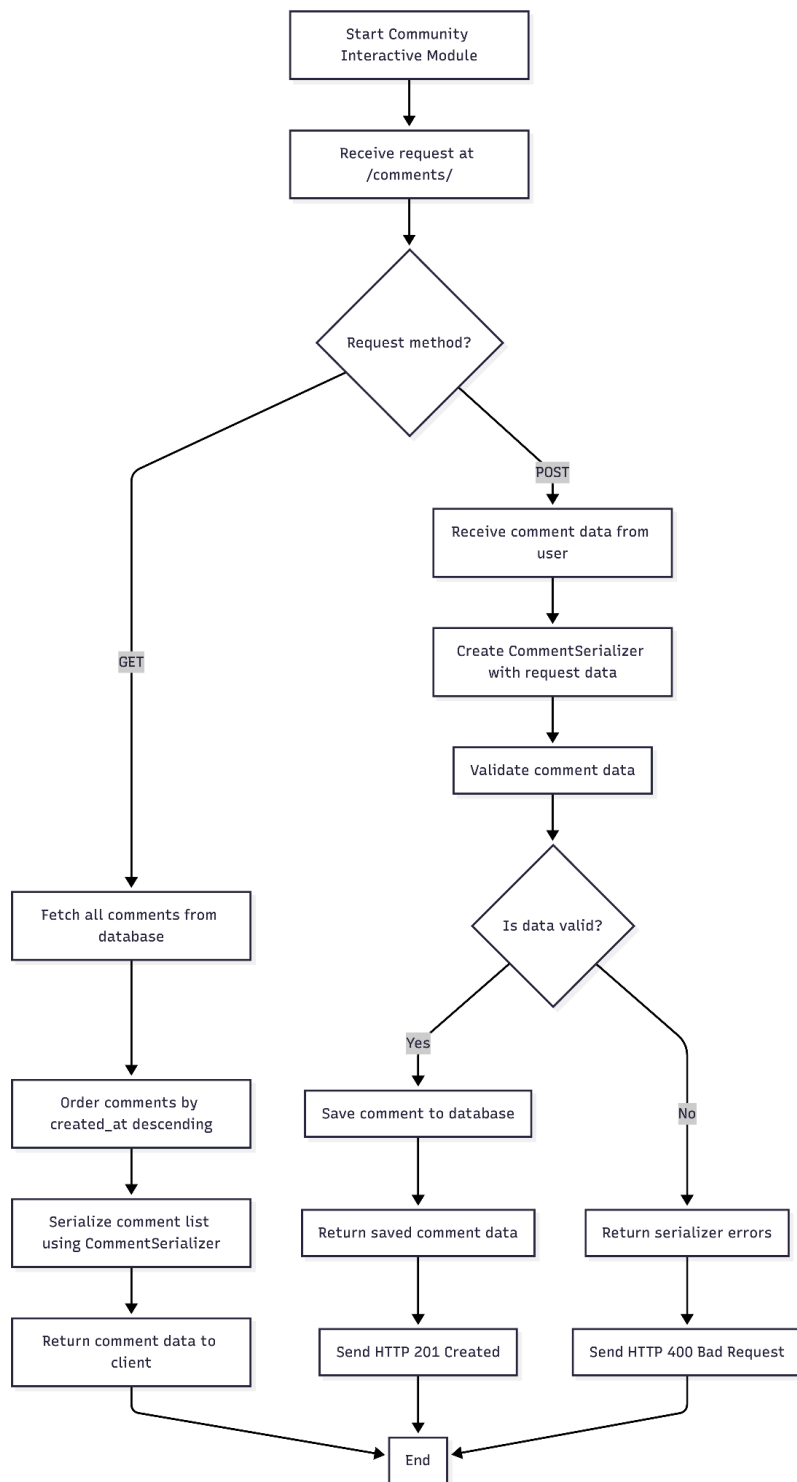


Fig. 2.4.2

5. Multi-Criteria Search Module (Success Criterion 5)

- Wireframe of GUI: Top search bar with a "Category" dropdown (News/Community).
- Function Description: Performs Fuzzy Search using SQL **LIKE** queries to find keywords in titles and content.

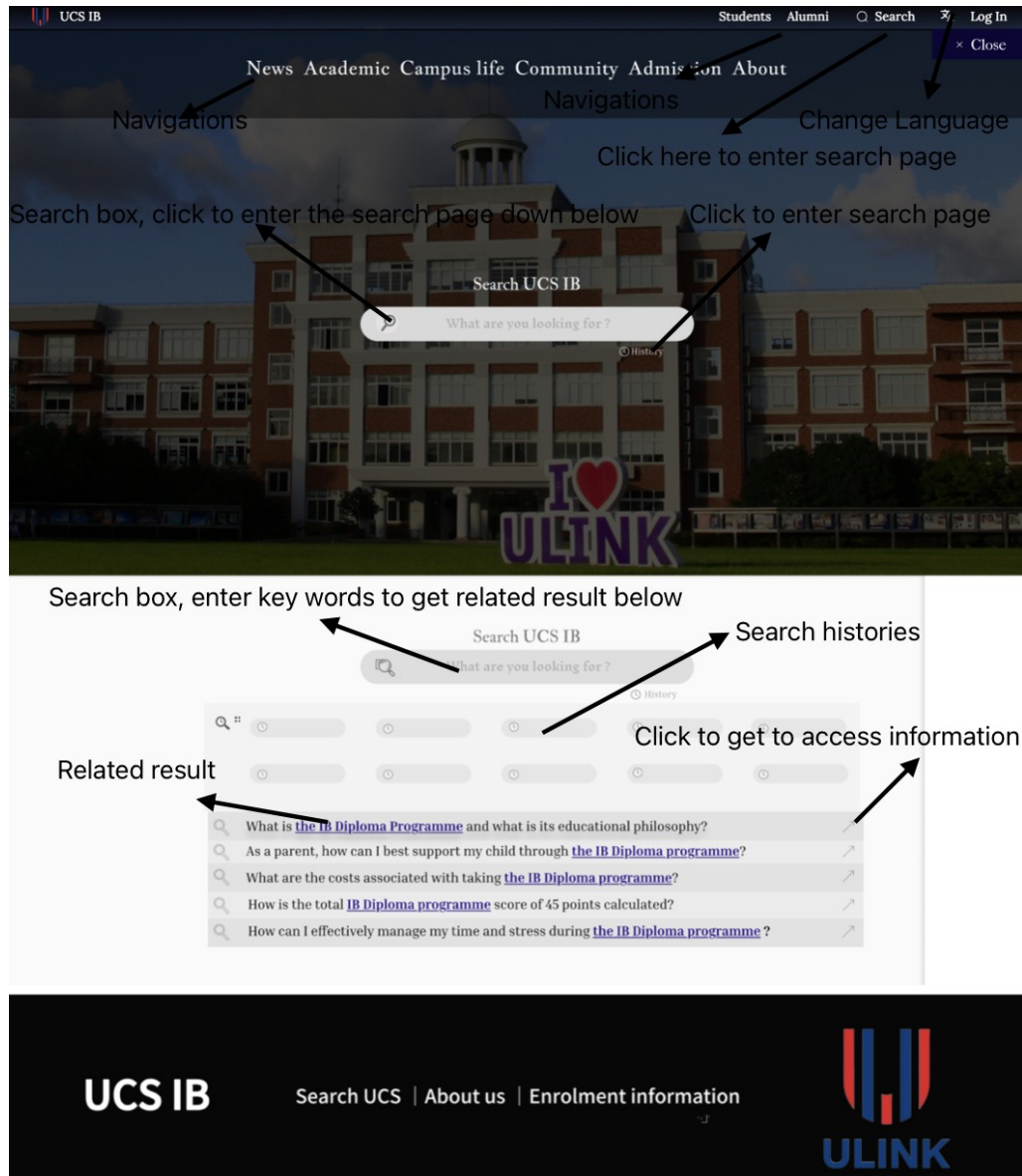


Fig. 2.5.1

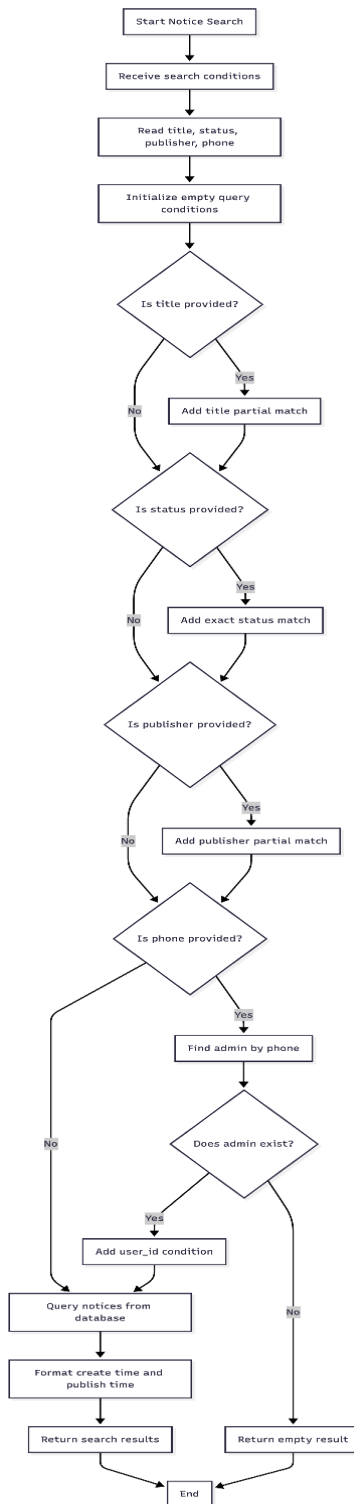


Fig. 2.5.2

6. Mobile Responsive Design (Success Criterion 6)

Responsive CSS media queries adapt the interface at widths of 575.98 pixels or below. Desktop navigation is replaced by a collapsible menu, multi-column content becomes full-width or vertical, and non-essential panels are hidden to prevent horizontal scrolling.

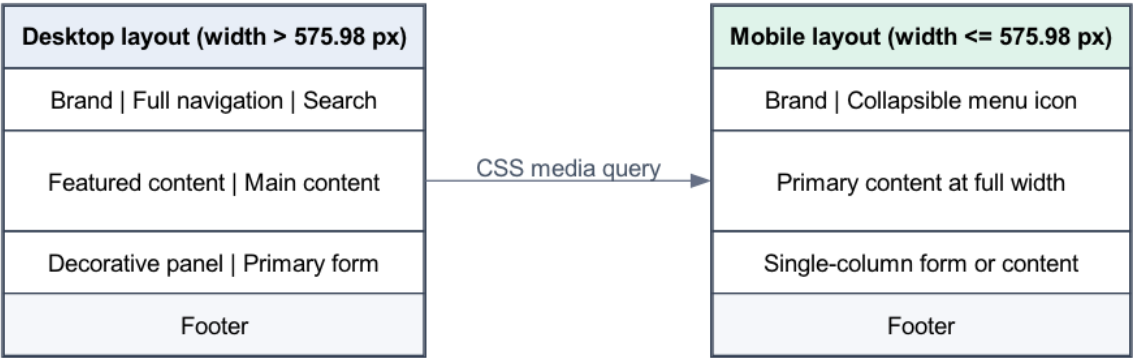


Fig. 2.6.1 Desktop and Mobile Layout Comparison

Element	Desktop design	Mobile design (<=575.98 px)
Navigation	Full horizontal menu and search	Collapsible menu; desktop search hidden
News	Featured area and notice list	Notice list expands to full width
Community	Side-by-side sections	Sections stack and images resize
Login	Decorative panel and form	Panel hidden; form uses full width

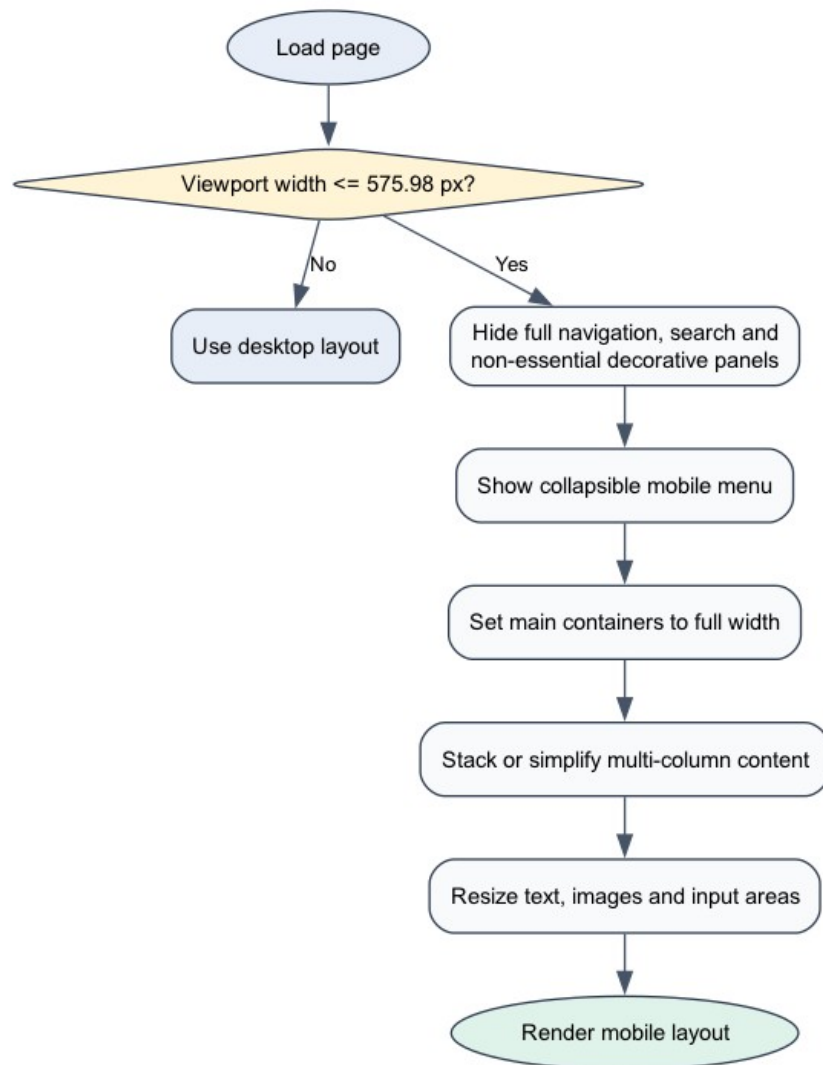


Fig. 2.6.2 Responsive Layout Decision Flow

3. UML Diagram

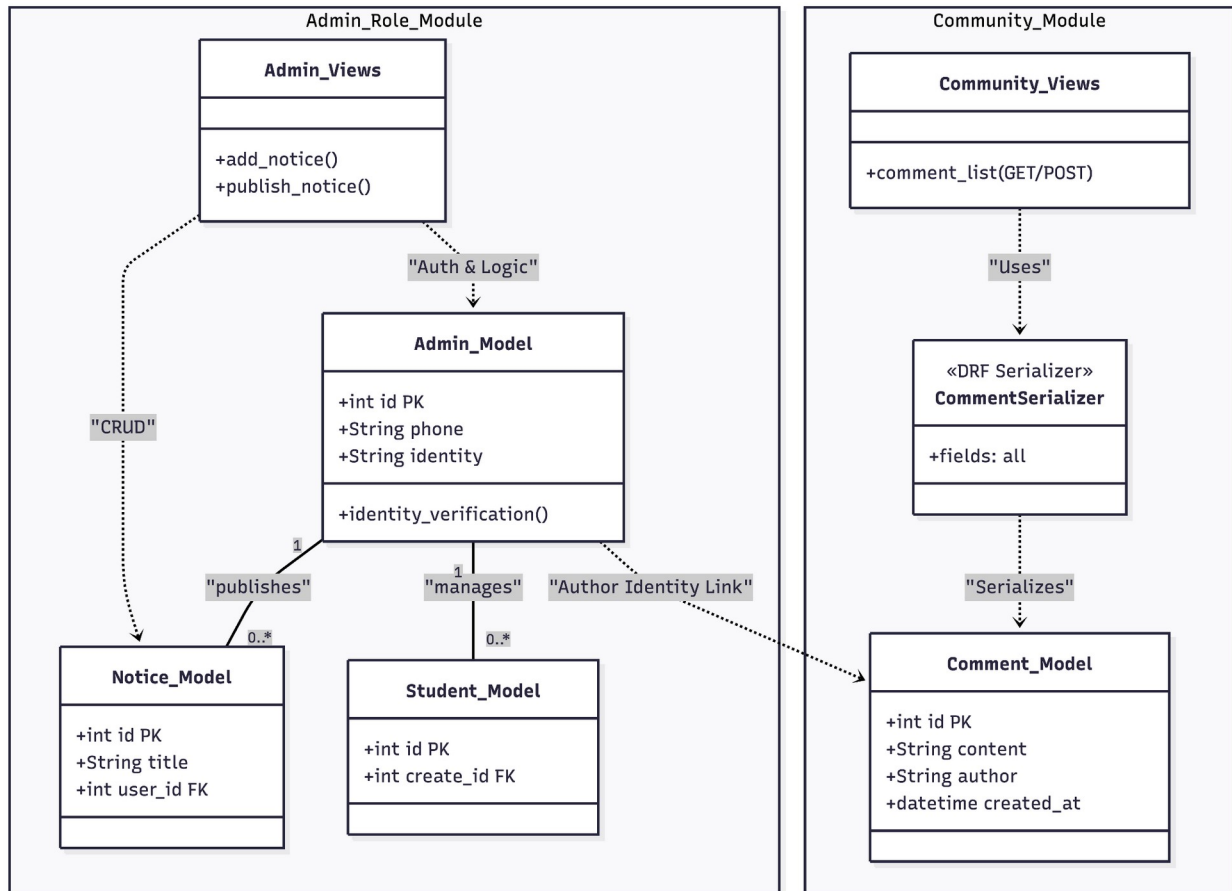


Fig. 3.1

Figure 3.1 maps request handlers to models and serializers. Notice ownership uses a database relationship, whereas comment identity is supplied from the authenticated JWT payload.

4. Testing Strategy

4.1 Methodology Description

Postman will simulate frontend API requests and verify JSON responses, HTTP status codes, JWT authorization and resulting MySQL changes for normal, boundary and invalid cases.

4.2 Representative Test Cases

4.2.1 User Authentication & Security (Related SC: #1)

Planned for execution through Postman

Test ID	Related SC	Test Data + Steps	Expected Result
T1	#1	Data: Username: miles_cai, Name: Miles, Phone: 13812345678, PW: SecurePass123, Confirm PW: SecurePass123. Steps: Open Register page → Enter all details → Check "User Agreement" → Click Sign Up.	[Registration Case] New user record is created in the admin_role_admin table; Password is MD5 hashed for security; System redirects to the Sign-In page.
T2	#1	Data: Phone: 13812345678, PW: SecurePass123. Steps: Open Sign-In page → Enter Phone and Password → Click Sign In.	[Login Case] System validates credentials; JWT Token is issued; User is redirected to the Dashboard; User name is displayed in the header.

4.2.2 Notice Management & CRUD (Related SC: #2)

Test ID	Related SC	Test Method	Expected Result
T3	#2	Create a notice with a .jpg cover image.	Image uploaded to Qiniu Cloud; Record saved in admin_notice_copy1.
T4	#2	Admin edits a notice title and clicks "Update".	DB updates immediately; Dashboard reflects new title.
T5	#3	Set notice publish_time to 1 min in future.	Asynchronous Thread auto-updates status to "Published" at the set time.

4.2.3 Community Comment Board (Related SC: #4)

Test ID	Related SC	Test Method	Expected Result
T6	#4	Student posts: "Is there a club meeting today?".	Comment saved in community_comment with the JWT-derived author name.
T7	#4	Attempt to post a comment with an empty string.	[Invalid Case] DRF returns 400 Bad Request; record blocked.
T8	#1	Student account attempts to access /admin_add.	[Security Case] Backend authorization blocks request; returns 403 Forbidden.

4.2.4 Global Search & Data Filtering (Related SC: #5)

Test ID	Related SC	Test Method	Expected Result
T9	#5	Search the keyword "Basketball" in "Announcements".	The system executes icon contains query; lists all matching notices.
T10	#5	Search for a non-existent string "abc_123".	[Boundary Case] GUI displays "No results found" message.

4.2.5 T12-T13. Mobile Compatibility (Related SC: #6)

Test ID	Related SC	Test Method	Expected Result
T12	#6	Method: Use Chrome DevTools to simulate iPhone 14 screen width.	[UI Case] Navigation bar collapses into a "Hamburger Menu"; layout switches to a single-column view.
T13	#6	Method: Test "Sign In" button on a mobile touchscreen device.	[UX Case] Input fields and buttons are appropriately sized for touch interaction; text remains readable.